SAECOIN — INCIDENT RESPONSE PLAN

Document: Incident_Response_Plan.md
Project: SAECOIN
Version: 1.0
Status: Draft — Implementation Pending
Security Priority: Critical
Applies To: Blockchain, Nodes, Mining, Wallets, APIs, Explorer, Infrastructure

---

1. PURPOSE

This document defines how the SAECOIN development and operations team should respond to security incidents, network failures, software vulnerabilities, and other critical operational problems.

The objectives are to:

- Detect incidents quickly.
- Protect users and network integrity.
- Limit financial and operational damage.
- Coordinate emergency responses.
- Release secure patches.
- Restore normal network operations.
- Document lessons learned.

This document does not authorize arbitrary consensus changes or guarantee that every incident can be resolved without disruption.

---

2. INCIDENT RESPONSE PRINCIPLES

All incident responses must follow these principles:

1. Protect user funds.
2. Preserve blockchain integrity.
3. Avoid unverified emergency changes.
4. Communicate honestly with users.
5. Preserve evidence for investigation.
6. Prioritize critical vulnerabilities.
7. Require appropriate review before releasing patches.
8. Document all major decisions.

Never hide a serious vulnerability or misleadingly describe an unresolved incident as fixed.

---

3. INCIDENT SEVERITY LEVELS

SEV-1 — Critical

Examples:

- Unauthorized coin creation.
- Consensus failure.
- Private-key theft affecting users.
- Critical signature verification vulnerability.
- Widespread invalid block acceptance.
- Active majority hashpower attack with significant impact.
- Compromised official release signing keys.

Required response:

- Immediate investigation.
- Emergency coordination.
- Protect affected infrastructure.
- Notify affected users and node operators.
- Prepare a verified remediation plan.

SEV-2 — High

Examples:

- Major network outage.
- Deep chain reorganization.
- Significant wallet vulnerability.
- Critical node crash affecting many operators.
- Widespread P2P denial-of-service attack.

Required response:

- Urgent investigation.
- Mitigation and monitoring.
- Timely user communication.
- Patch or operational recovery plan.

SEV-3 — Medium

Examples:

- Partial API outage.
- Explorer malfunction.
- Limited peer connectivity problems.
- Non-critical wallet defects.

Required response:

- Investigate.
- Record the issue.
- Schedule an appropriate fix.
- Monitor for escalation.

SEV-4 — Low

Examples:

- Minor interface defects.
- Documentation mistakes.
- Non-security performance issues.

Required response:

- Record and resolve through normal development procedures.

---

4. INCIDENT RESPONSE ROLES

During development, one person may perform multiple roles. However, responsibilities must remain clearly defined.

Role| Responsibility
Incident Coordinator| Coordinates response
Technical Lead| Investigates technical causes
Security Lead| Evaluates security impact
Node Operator| Manages affected nodes
Wallet Maintainer| Investigates wallet problems
Release Manager| Prepares verified releases
Communications Lead| Publishes user notices

For critical incidents, important changes should receive independent review whenever practical.

---

5. INCIDENT RESPONSE LIFECYCLE

Every major incident should follow these stages:

1. Detection.
2. Classification.
3. Containment.

4. Investigation.
5. Remediation.
6. Recovery.
7. Communication.
8. Post-incident review.

---

6. INCIDENT DETECTION

Potential detection sources:

- Node logs.
- Mining statistics.
- Block validation failures.
- Unusual chain reorganizations.
- Peer connection anomalies.
- Wallet security reports.
- API monitoring.
- User reports.
- Independent security researchers.

Monitor:

- Block production intervals.
- Network hashrate estimates.
- Difficulty changes.
- Invalid block counts.
- Reorganization depth.
- Node crashes.
- Abnormal transaction activity.
- Unexpected resource consumption.

Never log private keys or seed phrases for debugging.

---

7. BLOCKCHAIN CONSENSUS INCIDENT

Severity: SEV-1

Examples:

- Invalid blocks accepted by nodes.
- Different nodes calculate different UTXO states.
- Block reward validation fails.
- Unauthorized coin creation.
- Consensus implementation produces incompatible results.

Response Procedure

1. Confirm the consensus problem.
2. Identify affected software versions.
3. Determine whether the network has split.
4. Preserve relevant logs and block data.
5. Stop deploying unverified updates.
6. Assess whether normal transaction processing is safe.
7. Coordinate with node operators.
8. Develop a reviewed correction.
9. Test the correction against historical and adversarial test cases.
10. Publish verified upgrade instructions.

A consensus incident may require coordinated network recovery.

Do not invent a new chain-selection rule or manually rewrite blockchain history without a clearly documented, independently reviewed recovery decision.

---

8. MAJORITY HASHPOWER ATTACK

Severity: SEV-1

Possible indicators:

- Unexpected deep reorganizations.
- Repeated reversal of recent transactions.
- Unusual mining concentration.
- Abnormal block production.
- Evidence of transaction censorship.

Response Procedure

1. Verify the suspected attack using independent nodes.
2. Record affected block heights and transaction identifiers.
3. Warn exchanges and merchants to review payment acceptance.
4. Increase monitoring of confirmation depth.
5. Investigate mining distribution.
6. Publish an accurate network status notice.
7. Evaluate recovery options with independent technical review.

Do not claim that a specific confirmation count completely prevents majority hashpower attacks.

---

9. WALLET SECURITY INCIDENT

Severity: SEV-1 or SEV-2

Examples:

- Seed phrase exposure.
- Private-key leakage.
- Wallet encryption bypass.
- Malicious wallet update.
- Unauthorized transaction signing.

Response Procedure

1. Confirm the affected wallet version.
2. Identify the vulnerability and its scope.
3. Stop distributing affected wallet releases.
4. Publish a security warning through official channels.
5. Provide verified migration instructions if required.
6. Prepare a patched wallet.
7. Independently review the fix.
8. Test wallet backup and recovery.
9. Publish a signed release.
10. Continue monitoring reports.

Never ask users to send seed phrases or private keys to the development team.

If private keys may have been compromised, users must be given safe instructions for creating a new wallet and transferring funds using a trusted device and verified software.

---

10. NETWORK DENIAL-OF-SERVICE INCIDENT

Severity: SEV-2 or SEV-1

Examples:

- Connection flooding.
- Malformed message attacks.
- Excessive transaction requests.
- Resource exhaustion.
- Bootstrap infrastructure outage.

Response Procedure

1. Identify affected nodes.
2. Examine network traffic and resource usage.

3. Apply safe connection and request limits.
4. Block clearly abusive sources where appropriate.
5. Preserve legitimate peer connectivity.
6. Verify that block validation remains correct.
7. Release a reviewed patch if necessary.

Do not introduce arbitrary consensus changes merely to respond to a network-level attack.

---

11. NODE AND DATABASE FAILURE

Severity: SEV-2 or SEV-3

Examples:

- Node crashes.
- Database corruption.
- Disk exhaustion.
- Synchronization failure.
- Unexpected shutdown.

Response Procedure

1. Stop repeated unsafe restart attempts.
2. Preserve the affected database where possible.
3. Record the last known valid block height.
4. Check logs and storage health.
5. Restore from a verified backup or resynchronize from trusted peer data.
6. Validate recovered chain state.
7. Confirm correct peer synchronization.
8. Document the failure.

Never assume that a successful database startup proves blockchain state is valid.

---

12. SOFTWARE SUPPLY-CHAIN INCIDENT

Severity: SEV-1 or SEV-2

Examples:

- Compromised dependency.
- Malicious build artifact.
- Unauthorized source changes.
- Compromised GitHub account.
- Stolen release signing key.

Response Procedure

1. Identify affected repositories and releases.
2. Stop distributing suspicious binaries.
3. Secure repository access.
4. Revoke compromised credentials where appropriate.
5. Investigate dependency and build history.
6. Produce a clean, independently verified build.
7. Rotate compromised signing keys through a documented process.
8. Publish new checksums and release signatures.
9. Notify affected users.

Signing-key compromise must be treated as a critical trust incident.

---

13. API AND EXPLORER INCIDENT

Severity: SEV-2 or SEV-3

Examples:

- Unauthorized API access.
- Incorrect balance display.
- Explorer data manipulation.
- Service outage.
- API abuse.

Response Procedure

1. Identify the affected service.
2. Restrict compromised access.
3. Verify data against independent validating nodes.
4. Restore service from trusted software and data.
5. Publish a correction if users received incorrect information.
6. Review authentication and access controls.

An explorer outage does not necessarily mean the blockchain itself is offline.

---

14. INCIDENT COMMUNICATION

Official notices should include:

- Incident identification.
- Date and time.

- Severity.
- Affected software or services.
- Known impact.
- Recommended user actions.
- Current investigation status.
- Verified recovery instructions.
- Next update information.

Do not publish unverified technical explanations.

Do not expose private user information or sensitive operational credentials.

For critical incidents, use multiple trusted communication channels.

---

15. EMERGENCY RELEASE PROCEDURE

Emergency releases must follow a controlled process.

1. Identify the vulnerability.
2. Create a dedicated fix branch.
3. Implement the minimum necessary correction.
4. Add regression tests.
5. Run relevant unit and integration tests.
6. Run consensus compatibility tests if applicable.
7. Obtain independent review for critical changes.
8. Build from a trusted environment.
9. Sign the release.
10. Publish checksums and upgrade instructions.
11. Monitor adoption and reported problems.

Consensus changes require additional coordination because incompatible node versions can split the network.

---

16. EVIDENCE PRESERVATION

During major incidents, preserve:

- Relevant node logs.
- Software version information.
- Block hashes.
- Transaction identifiers.
- Network timestamps.
- Error reports.
- Build metadata.

- Security alerts.
- Relevant configuration files with secrets removed.

Never publish private keys, seed phrases, passwords, access tokens, or other credentials as incident evidence.

---

17. POST-INCIDENT REVIEW

After every SEV-1 or SEV-2 incident, prepare a report containing:

1. Incident summary.
2. Timeline.
3. Root cause.
4. Affected systems.
5. User impact.
6. Financial or operational impact, if known.
7. Actions taken.
8. Resolution details.
9. Remaining risks.
10. Preventive improvements.

The review should focus on improving systems and procedures rather than assigning blame.

Update the Threat Model and relevant technical documents when necessary.

---

18. INCIDENT RESPONSE TESTING

Before mainnet, simulate:

- Invalid block acceptance.
- Wallet key exposure.
- Node database corruption.
- Network partition.
- DDoS conditions.
- Compromised dependency.
- Release signing-key compromise.
- Incorrect explorer balances.

The team must demonstrate that it can detect, investigate, communicate, and recover from these scenarios.

---

19. MAINNET INCIDENT READINESS

Before mainnet launch, SAECOIN must have:

- A documented security contact.
- A designated incident coordinator.
- Secure release signing procedures.
- Tested backup and recovery processes.
- A critical vulnerability reporting process.
- Emergency communication channels.
- A tested patch release procedure.
- A documented consensus incident process.
- A public security advisory process.

---

20. FINAL PRINCIPLE

Security incidents are possible in every software and blockchain project.

SAECOIN must prepare for incidents before they happen.

The response must protect users, preserve evidence, avoid rushed protocol changes, and communicate honestly.

A secure project is not one that claims it will never fail. It is one that prepares to detect, contain, investigate, and recover from failures responsibly.

END OF DOCUMENT